



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **informatyka**

Mateusz Kowal

nr albumu: 318683

Środowisko programowalnego bezzałogowego statku powietrznego z użyciem mikrokomputera Raspberry Pi

A programmable unmanned aerial vehicle environment
based on a Raspberry Pi microcomputer

Praca licencjacka

napisana w Katedrze Oprogramowania Systemów Informatycznych

Instytutu Informatyki i Matematyki UMCS

pod kierunkiem **dr hab. Beaty Byliny**

Lublin 2026

Spis treści

Wstęp	5
1 Wprowadzenie do bezzałogowych statków powietrznych	7
1.1 Podstawowe pojęcia	7
1.2 Zadania i ograniczenia komputera pokładowego	7
1.2.1 Podstawowe zadania komputera pokładowego	7
1.2.2 Opcjonalne zastosowania autopilota	8
1.3 Rozszerzenie możliwości BSP dzięki komputerom asystującym	8
2 Szczegóły techniczne zbudowanego środowiska	11
2.1 Lista komponentów i ich specyfikacja	11
2.1.1 Rama BSP	11
2.1.2 Napęd	11
2.1.3 Zewnętrzny moduł GNSS	14
2.1.4 Układ komunikacji z pilotem	14
2.1.5 Komputer pokładowy	14
2.1.6 Komputer asystujący	15
2.1.7 Źródło zasilania	15
2.2 Specyfikacja drona	15
3 Zastosowanie środowiska	17
3.1 Wstępne założenia	17
3.2 Ostrzeżenie przed zbliżającą się przeszkodą	20
3.3 Kod programu	21
3.4 Dynamiczne ustawianie odległości progu aktywacji	24
3.5 Testy	28
3.5.1 Sprawdzenie poprawności działania systemu	28
3.5.2 Pobór prądu	29
3.5.3 Dokładność pomiarów odległości	29
3.5.4 Czas reakcji systemu	29

3.6	Alternatywne rozwiązania	30
4	Potencjalne ścieżki rozwoju projektu	33
4.1	Półautonomiczny system zapobiegania kolizjom	33
4.2	Wielokierunkowy monitoring	33
4.3	Sieć neuronowa rozpoznawanie obiektów z obrazu kamery	33
4.4	mapa 3d lidar	33
	Bibliografia	33

Wstęp

Tu treść wstępu (który piszemy na końcu, po napisaniu całej pracy).

Rozdział 1

Wprowadzenie do bezzałogowych statków powietrznych

1.1 Podstawowe pojęcia

Bezzałogowy statek powietrzny (skrót BSP) to kategoria pojazdów poruszających się w powietrzu do której zaliczane są samoloty, śmigłowce i pojazdy wielowirnikowe (w tym drony), które odbywają loty bez pilota na pokładzie i bez możliwości zabierania pasażerów. Sterowanie może się odbywać zdalnie przez pilota naziemnego lub przez komputer, jeśli pojazd jest autonomiczny [2].

Komputer pokładowy jest urządzeniem niezbędnym do działania BSP. Jego głównym zadaniem jest sterowanie silnikami pojazdu oraz otrzymywanie i przetwarzanie sygnałów docierających z aparatury sterującej. W tym przypadku rolę komputera pokładowego pełni Pixhawk 6C.

Komputer asystujący to komputer znajdujący się na pokładzie BSP, połączony z komputerem pokładowym. Jego rolą jest wykonywanie zadań zbyt skomplikowanych obliczeniowo by móc powierzyć je komputerowi pokładowemu lub wymagają elementów przez komputer pokładowy nieposiadanych i niewspieranych. Komputerem asystującym w przypadku tego projektu jest Raspberry Pi 5B.

1.2 Zadania i ograniczenia komputera pokładowego

1.2.1 Podstawowe zadania komputera pokładowego

Rolą każdego komputera pokładowego jest przede wszystkim sterowanie silnikami pojazdu którego jest częścią. Komputer pokładowy musi więc posiadać odpowiednie interfejsy do komunikacji z silnikami oraz interfejs do otrzymywania poleceń. W przy-

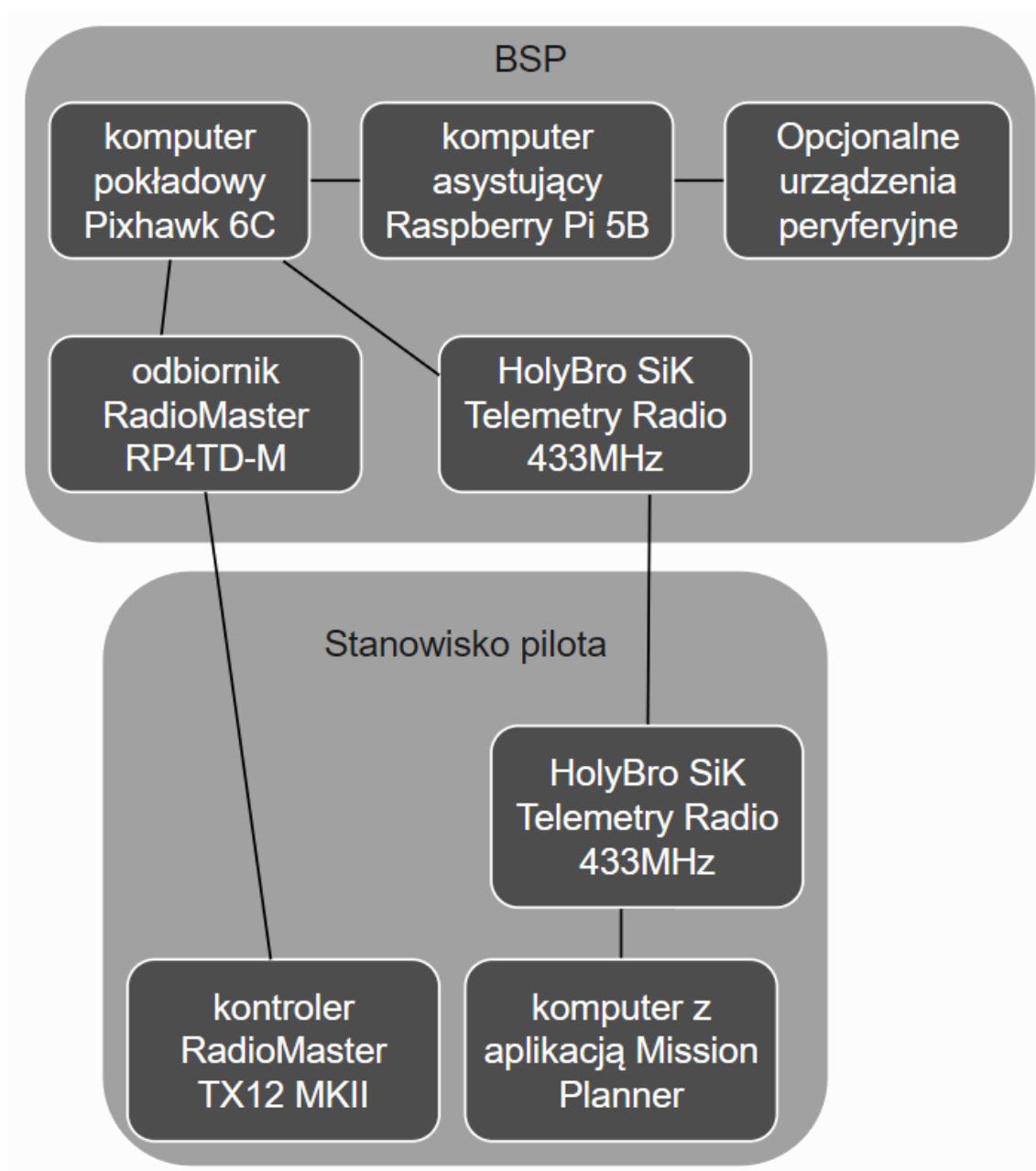
padku komputerów pokładowych używanych w BSP wielowirnikowych niezbędny jest również żyroskop, dzięki któremu komputer pokładowy zna swoją pozycję i jest w stanie utrzymać pojazd poziomo lub odpowiednio odchylić go zgodnie z poleceniami które otrzyma.

1.2.2 Opcjonalne zastosowania autopilota

W zależności od modelu komputera pokładowego oraz zainstalowanego na nim oprogramowania może on dodatkowo obsługiwać wiele urządzeń peryferyjnych i posługiwać się bardziej złożoną logiką. Przeciętny komputer pokładowy dostępny na rynku cywilnym oferuje możliwość podłączenia zewnętrznego modułu GPS czy kamery, a także samodzielnie przechowywać i wykonywać polecenia pozwalające na lot autonomiczny. Standardem jest również możliwość ustawienia procedury bezpieczeństwa (*ang. failsafe*), która decyduje jak zachowa się pojazd w razie utraty podłączenia z kontrolerem naziemnym lub jakimkolwiek innym urządzeniem wydającym polecenia komputerowi pokładowemu.

1.3 Rozszerzenie możliwości BSP dzięki komputerom asystującym

Jeśli użytkownik chce wykorzystać swój BSP w bardziej nietypowy sposób, na drodze mogą stanąć mu ograniczenia sprzętowe. Komputery pokładowe oferują zazwyczaj niewielką moc obliczeniową i ograniczoną ilość i różnorodność gniazd do podłączenia urządzeń peryferyjnych. Jednym ze sposobów przystosowania BSP do naszych wymagań jest użycie komputera asystującego który rozszerzy możliwości jednostki. Przykładowo, użytkownik może chcieć użyć swojego BSP do monitorowania dużego obszaru. W tym celu chciałby zamontować kamery z wielu stron, jednak użyty przez niego komputer pokładowy ma możliwość podpięcia tylko jednej kamery. Użycie komputera asystującego może być wtedy rozwiązaniem problemu. W zależności od użytego komputera asystującego można użyć nawet tak złożonych obliczeniowo programów jak sztuczna sieć neuronowa analizująca dane z urządzeń wejściowych i wydająca polecenia jednostce. Sposób połączenia komputera asystującego oraz połączeń dających użytkownikowi kontrolę nad dronem i informacje zwrotne zostały przedstawione na rysunku [1.1](#).



Rysunek 1.1: Schemat wybranych komponentów jednostki i połączenia między nimi

Rozdział 2

Szczegóły techniczne zbudowanego środowiska

2.1 Lista komponentów i ich specyfikacja

Większość gotowych bezzałogowych pojazdów dostępnych na rynku to zamknięte systemy a ingerencje i modyfikacje są bardzo utrudnione, zarówno sprzętowo jak i w oprogramowaniu. Samodzielna budowa jednostki z użyciem kupionych oddzielnie komponentów oraz oprogramowania open-source daje dużo większe możliwości w przystosowaniu platformy do własnych wymagań i preferencji.

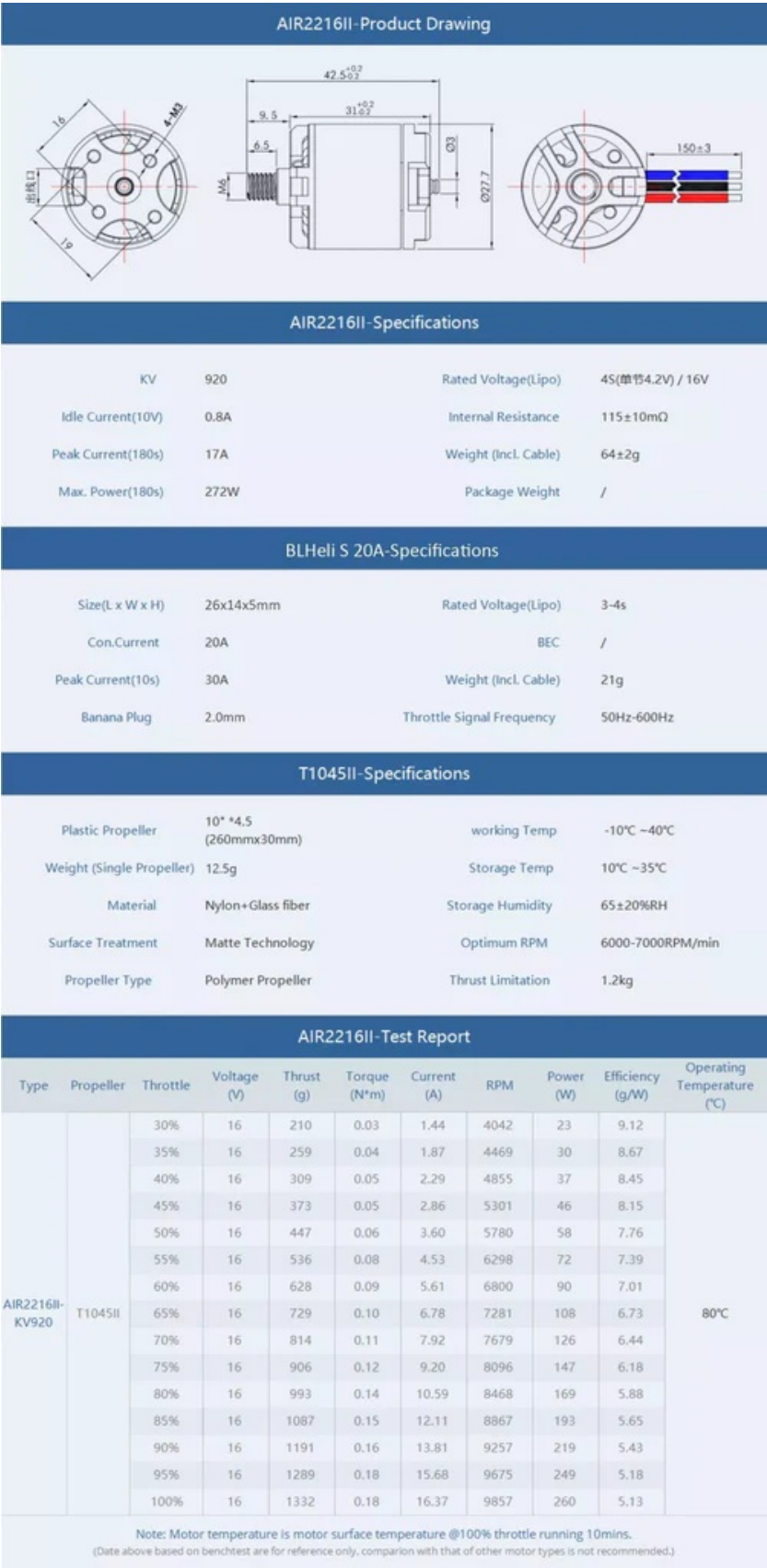
2.1.1 Rama BSP

Rama użyta do zbudowania jednostki to Holybro S500 v2. Oferuje wbudowaną w konstrukcję kadłuba rozdzielnicę napięcia (*ang. power distribution board*) dostarczającą energię elektryczną z akumulatora do silników. Miejscem na montaż silników są cztery ramiona wykonane z nylonu poliamidowego, wzmocnione w środku poprzez dodanie prętów z włókna węglowego. Całość opiera się na podwoziu z rurek z włókna węglowego połączonych plastikowymi łącznikami.

2.1.2 Napęd

Pojazd napędzany jest przez cztery silniki bezszczotkowe AIR2216II osiągające do 272 W. Silniki bezszczotkowe wymagają zasilania prądem zmiennym trójfazowym, dlatego do ich działania potrzebne są moduły elektronicznej kontroli prędkości (*ang. electronic speed controller*). Moduły te otrzymują zasilanie prądem stałym z akumulatora oraz sygnał z komputera pokładowego sterujący prędkością obracania się silników. Następnie generuje prąd zmienny trójfazowy który wywołuje obrót silników. Moduły

ESC zastosowane w zbudowanym pojeździe to BLHeli S 20A. Jednostka została wyposażona w śmigła T1045II o rozpiętości 260 milimetrów i szerokości do 30 milimetrów. Specyfikacja każdego z elementów napędu została pokazana na rysunku [2.1](#).



Rysunek 2.1: Dokumentacja techniczna AIR2216II, BLHeli S 20A, T1045II. Źródło: [3]

2.1.3 Zewnętrzny moduł GNSS

Zastosowany w jednostce zewnętrzny moduł GNSS to M10 od firmy Holybro. Oferuje jednocześnie otrzymywanie informacji od czterech największych systemów satelit geolokacyjnych: europejskich satelit Galileo, amerykańskich GPS, rosyjskich GLO-NASS i chińskich Bei Dou. Oferowana dokładność wyznaczonej pozycji to 2.0 m CEP, co oznacza że co najmniej 50% pomiarów wskaże pozycję oddaloną o 2 metry lub mniej od rzeczywistej pozycji [4].

2.1.4 Układ komunikacji z pilotem

BSP został przygotowany do utrzymywania dwóch rodzajów komunikacji z pilotem. Podstawowym kanałem komunikacji jest kontroler RadioMaster TX12 MKII pozwalający na zdalne sterowanie dronem wyposażonym w odbiornik radiowy RadioMaster RP4TD-M używając systemu ExpressLRS zapewniającego niezawodność i stabilność łącza nawet na dużych dystansach. Drugim kanałem komunikacji jest system telemetry Holybro SiK Telemetry Radio V3 w wersji 433 MHz. Moduł podłączony do komputera osobistego znajdującego się na ziemi w pobliżu pilota pozwala na użycie aplikacji Mission Planner—oprogramowania pozwalającego między innymi na śledzenie BSP na mapie czy odczyt bieżących danych [1].

2.1.5 Komputer pokładowy

Rolę komputera pokładowego pełni Pixhawk 6C. Wyposażony jest w dwa procesory: STM32H743 — 480MHz Cortex-M7 odpowiedzialny za działanie oprogramowania zainstalowanego na urządzeniu, odczyt parametrów z sensorów, przetwarzanie informacji i przekazywanie poleceń dotyczących silników do drugiego procesora, w tym przypadku STM32F103 — 72 MHz Cortex-M3. Zadaniem drugiego procesora jest generowanie sygnałów PWM na podstawie otrzymanych danych i przekazywanie ich do modułów ESC. W przypadku wystąpienia problemów z pierwszym procesorem jest w stanie samodzielnie generować sygnał sterujący silnikami, jednak przez brak dostępu do czujników takich jak żyroskopy czy akcelerometry jego możliwości ograniczają się do utrzymania kręcenia się śmigieł w celu spowolnienia upadku i minimalizacji uszkodzeń przy lądowaniu awaryjnym.

Do poprawnego sterowania pojazdem wymagane są również odpowiednie czujniki. Pixhawk 6C posiada dwa urządzenia łączące w sobie akcelerometry i żyroskopy: ICM-42688-P i BMI055. Użycie dwóch urządzeń i porównanie ich odczytów ze sobą pozwala na wykrycie niedokładności lub uszkodzenia jednego z nich i zapobiega destabilizacji drona w przypadku awarii. W razie uszkodzenia komputer pokładowy

może wdrożyć procedurę bezpieczeństwa i bezpiecznie ją wykonać polegając na pozostałym działającym czujniku. Pixhawk 6C wyposażony jest również w magnetometr IST8310 pozwalający na określenie jego pozycji na podstawie pola magnetycznego Ziemi oraz barometr MS5611 pozwalający obliczyć wysokość nad ziemią na podstawie różnicy ciśnień [8]. Oprogramowaniem działającym na komputerze pokładowym jest ArduCopter w wersji 4.6.2.

2.1.6 Komputer asystujący

Asystentem dla Pixhawk 6C jest mikrokomputer Raspberry Pi 5B. Posiada on czterordzeniowy procesor Arm Cortex-A76, 8 GB pamięci RAM oraz szereg sposobów na podłączenie urządzeń peryferyjnych, np. cztery gniazda USB czy 40 pinów GPIO. Zainstalowany na nim system operacyjny to Raspberry Pi OS będący portem Debiana Trixie.

2.1.7 Źródło zasilania

BSP zasilany jest za pomocą akumulatora litowo-jonowego Gens Ace G-Tech o pojemności 5000 mAh, napięciu 14.8 V i wadze 439 g. Za zasilanie komputera asystującego odpowiedzialny jest powerbank z baterią litowo-polimerową o pojemności 10000 mAh, napięciu 5 V, mocy maksymalnej 22.5 W i wadze 235 g. Możliwe jest zasilenie Raspberry Pi akumulatorem drona, wiąże się to jednak z ingerencją w gotowy układ zasilania oraz koniecznością obniżenia napięcia z 14.8 V do 5 V. Problem stanowią również silniki elektryczne napędzające jednostkę, ich zużycie prądu zmienia się bardzo dynamicznie co stwarza ryzyko spadku napięcia zasilającego komputer asystujący poniżej wymaganych 5 V. W przypadku Raspberry Pi spadek napięcia poniżej 4.6 3V ($\pm 5\%$) może skutkować obniżoną wydajnością komputera, nieprawidłowym działaniem podłączonych urządzeń peryferyjnych lub do nieprzewidywalnego zachowania urządzenia czy uszkodzenia danych na karcie pamięci. [6] Z uwagi na potencjalne problemy i konieczność ingerencji w gotowy układ zasilania zastosowane zostało oddzielne źródło prądu dla komputera asystującego.

2.2 Specyfikacja drona

Waga jednostki, wliczając w to komputer asystujący i jego źródło zasilania, wynosi 1715 g. Na podstawie wagi, specyfikacji silników [3] oraz akumulatora można oszacować średni możliwy czas lotu. Aby dron zawisł w bezruchu, każdy z czterech silników musi wytworzyć ok. 428.75 g ciągu. Dzięki dokumentacji wiadomo że jeden silnik z za-

stosowanym dołączonym w zestawie śmigłem potrzebuje 2.86 A do wytworzenia 373 g ciągu i 3.60 A dla 447 g ciągu. Prąd potrzebny do wytworzenia dokładnie 428.75 g ciągu może być obliczony z użyciem interpolacji liniowej:

$$I = I_1 + \frac{T - T_1}{T_2 - T_1}(I_2 - I_1),$$

$$I = 2.86 + \frac{428.75 - 373}{447 - 373}(3.60 - 2.86),$$

$$I \approx 3.42 \text{ A}.$$

Jeden silnik potrzebuje około 3.42 A, co dla czterech silników daje łącznie 13.67 A. Używając akumulatora o pojemności 5000 mAh daje to teoretyczne 21.95 minut lotu. Biorąc pod uwagę dodatkowe zużycie prądu przez komputer pokładowy oraz zwiększone zużycie prądu przy wznoszeniu i manewrowaniu, bezpieczniejszym będzie przyjęcie marginesu błędu i określenie czasu lotu na około 19–20 minut. Oszacowanie maksymalnego czasu działania Raspberry Pi jest dużo trudniejsze, zależy to w dużym stopniu od złożoności programów na niej uruchomionych. Artykuł ogłaszający wypuszczenie na rynek Raspberry Pi 5 w wersji z 16 GB pamięci RAM określa zużycie w spoczynku na 2–3 W [5], zakładając zużycie 3 W przy 10000 mAh i 5 V daje to około 16.7 godzin działania. Trzeba jednak wziąć pod uwagę że to ogłoszenie nie jest tak wiarygodnym źródłem jak oficjalna dokumentacja, która w tym przypadku nie wspomina o poborze mocy w stanie spoczynku. Pobór prądu w praktyce będzie też większy ze względu na uruchomione na mikrokomputerze programy. Opisane dalej testy wykazały że pobór mocy w stanie spoczynku wahają się między 3.177 W a 3.484 W, większość odczytów była zbliżona do 3.281 W. Przy parametrach zastosowanego zasilania szacowany czas działania to 11 godzin i 15 minut.

Rozdział 3

Zastosowanie środowiska

3.1 Wstępne założenia

Komputer Raspberry Pi jest połączony z Pixhawk 6C za pomocą USB. W katalogu `~/uavproject` znajduje się wirtualne środowisko python oraz wszystkie programy współpracujące z dronem. Komunikacja z dronem możliwa jest dzięki bibliotece DroneKit 2.9.2. Użycie tej biblioteki ogranicza wersję pythona do nie nowszej niż 3.9.18, ponieważ od jakiegoś czasu biblioteka są nieutrzymywane i nie współpracuje z nowszymi wersjami pythona. Za pomocą komendy `source uavproject/venv/bin/activate` uruchamiane jest wirtualne środowisko. Wewnątrz środowiska można testować napisane programy i instalować wymagane biblioteki. Pisanie programu najlepiej zacząć używając szablonu 3.1.

Listing 3.1: Szablon programu działającego na komputerze asystującym

```
1 from dronekit import connect, APIException
2 import os
3 import time
4
5 DRONE_PORT = """/dev/serial/by-id/usb-Holybro_Pixhawk6C_
6             450046001751333337363133-if00"""
7
8 def connect_pixhawk():
9     print("Connecting to Pixhawk...")
10
11     while True:
12         try:
13             vehicle = connect(
14                 DRONE_PORT,
15                 baud=115200,
16                 wait_ready=True,
17                 heartbeat_timeout=60
```

```
18         )
19         print("Connected to Pixhawk")
20         return vehicle
21
22     except APIException as e:
23         print(f"No heartbeat yet: {e}")
24         time.sleep(3)
25
26     except Exception as e:
27         print(f"Connection failed: {e}")
28         time.sleep(3)
29 def main():
30     vehicle = None
31
32     while not os.path.exists(DRONE_PORT):
33         time.sleep(1)
34     try:
35         vehicle = connect_pixhawk()
36
37         vehicle.message_factory.statustext_send(
38             mavutil.mavlink.MAV_SEVERITY_ALERT,
39             b"Companion computer connected"
40         )
41         vehicle.flush()
42
43         #miejsce na kod
44
45     except KeyboardInterrupt:
46         print("\nProgram interrupted by user.")
47
48     finally:
49         print("Closing connections...")
50
51         if vehicle:
52             vehicle.close()
53             print("Drone connection closed")
54
55 if __name__ == "__main__":
56     main()
57
```

Szablon obejmuje poprawne otwarcie połączenia z komputerem pokładowym za pomocą funkcji `connect_pixhawk()`. Funkcja tworzy połączenie i nasłuchuje sygnału *heartbeat*. Sygnał ten jest wysyłany przez komputer pokładowy w momencie gdy ten jest uruchomiony i gotowy do pracy. Po otrzymaniu sygnału połączenie jest zwracane

i może być użyte do komunikacji z dronem w dalszej części programu. Program, używając otwartego połączenia, wywołuje wysłanie wiadomości przez komputer Pixhawk o pomyślnym podłączeniu komputera asystującego. Wiadomość może być odczytana przez pilota mającego połączenie z jednostką dzięki aplikacji Mission Planner. Szablon uwzględnia też poprawne zamknięcie połączenia, również w przypadku jeśli działanie programu zakończy się poprzez przerwanie skrótem klawiszowym lub błąd programu. Bez zamknięcia połączenia każda kolejna próba uruchomienia programu zakończy się błędem, ponieważ urządzenie bez poprawnie zamkniętego połączenia będzie oznaczone jako zajęte i niedostępne.

Loty dronem najczęściej nie odbywają się w warunkach domowych, dlatego programy powinny się wykonywać automatycznie, bez potrzeby użycia klawiatury i monitora. W tym celu wykorzystany został skrypt 3.2 zapisany w `/etc/systemd/system/autorunuav.service`. Po uruchomieniu Raspberry Pi skrypt poczeka aż system będzie w pełni uruchomiony, po czym wykona program określony w parametrze `ExecStart` przy użyciu wirtualnego środowiska. W przypadku jeśli program przestanie działać skrypt odczeka pięć sekund i uruchomi go ponownie. Po każdej zmianie automatycznie uruchamianego programu lub jakichkolwiek innych zmian w skrypcie należy odświeżyć systemd komendą `sudo systemctl daemon-reload` oraz uruchomić skrypt za pomocą `sudo systemctl start autorunuav.service`. Wyjście programu można zobaczyć używając komendy `journalctl -u autorunuav.service -f`.

Listing 3.2: Skrypt wykonujący program po uruchomieniu komputera asystującego

```
1 [Unit]
2 Description=UAV Companion Autostart
3 After=multi-user.target
4
5 [Service]
6 Type=simple
7 User=uavcompanion
8 Group=uavcompanion
9
10 WorkingDirectory=/home/uavcompanion/uavproject
11 ExecStart=/home/uavcompanion/uavproject/venv/bin/python -u program.py
12
13 Restart=on-failure
14 RestartSec=5
15
16 StandardOutput=journal
17 StandardError=journal
18
```

```
19 [Install]
20 WantedBy=multi-user.target
21
```

3.2 Ostrzeżenie przed zbliżającą się przeszkodą

W celu przetestowania zbudowanej platformy i zaprezentowania jednej z możliwości jej wykorzystania zbudowano system ostrzegania przed kolizją. Wykrywanie przeszkód odbywa się z użyciem trzech ultradźwiękowych czujników odległości HC-SR04 [9]. Zasilanie 5 V oraz uziemienie są połączone równolegle do pinów w Raspberry Pi. Piny TRIG w czujnikach, wywołujące pomiar, są podłączone bezpośrednio do pinów GPIO. Wyjście ECHO czujników nie może być podłączone do Raspberry Pi ze względu na różnicę w napięciach. W czujnikach HC-SR04 dla sygnału wychodzącego przez pin ECHO stan wysoki to 5 V. Takie napięcie jest odpowiednie dla mikrokomputera Arduino, jednak stan wysoki dla pinów w Raspberry Pi to 3.3 V. Aby zapobiec uszkodzeniom należy obniżyć napięcie wychodzące z czujnika do bezpiecznych 3.3 V, co zostało osiągnięte z użyciem konwertera poziomów logicznych. Zaprojektowane i wydrukowane na drukarce 3D uchwyty pozwalają na zamontowanie czujników i konwertera na dronie oraz regulację ich pozycji w poziomie. Program 3.3 wywołuje wysłanie ostrzeżenia do pilota w przypadku jeśli wykryta zostanie przeszkoda oraz dron zbliża się do niej z odpowiednią prędkością. Zmienne globalne na początku programu określają między innymi minimalną odległość i prędkość przy których program powinien wywołać ostrzeżenie, wymaga również określenia kątów pod jakimi ustawione są czujniki. Wartość 0 oznacza czujnik zwrócony na wprost, wartości ujemne oznaczają odchylenie w lewo a wartości dodatnie oznaczają odchylenie w prawo. Dla wygody użytkownika kąty wyrażone są w stopniach. Po ustanowieniu połączenia z komputerem pokładowym program w nieskończonej pętli odczytuje odległości z czujników, zostawiając przerwę między odczytami równą wartości określonej w zmiennej GAP. W przypadku użycia wielu czujników obok siebie istnieje ryzyko że po wywołaniu pomiaru u jednego z czujników, przejdziemy do drugiego zanim poprzedni zakończy pomiar. W takiej sytuacji może się zdarzyć że fala ultradźwiękowa wysłana przez poprzedni czujnik dotrze do kolejnego, co skutkować będzie zaburzonym pomiarem i fałszywie krótkim zmierzonym dystansem. Ryzyko wystąpienia takiego problemu jest największe gdy czujniki zwrócone są w tę samą stronę, w przypadku zbudowanego systemu ostrzegania ryzyko zaburzenia pomiaru z tego powodu jest znikome. Mimo wszystko pozostawienie odstępu jest wskazane, a odstęp 50ms nie jest wystarczająco długi aby uczynić program zawodnym przez zbyt rzadkie pomiary. Każda wartość pomiaru zwracana jest w postaci liczby

zmiennoprzecinkowej pomiędzy 0 a 1, gdzie 0 oznacza 0 cm a 1 oznacza maksymalną odległość pomiaru określaną w zmiennej `MAX_DIST`. Wartość pomiaru jest mnożona przez 100 i maksymalną odległość pomiaru w celu uzyskania wartości w centymetrach. Komputer Pixhawk dostarcza programowi informacji o prędkości drona i jego odchyleniu od kierunku północnego. Prędkość drona jest wyrażona w postaci trzech wartości: prędkości w kierunku północnym, wschodnim i w dół. Dzięki odchyleniu od kierunku północnego obliczana jest prędkość względem BSP. Program następnie oblicza prędkość względem danego czujnika, wykorzystując określony wcześniej kąt jego odchylenia względem przodu. Mając te dane program porównuje odczytaną z czujników odległość z określonym progiem aktywacji oraz obecną prędkość w kierunku w który skierowany jest czujnik z minimalną prędkością aktywującą ostrzeżenie. Jeśli odległość jest mniejsza a prędkość większa niż punkty aktywacji, program wywoła wysłanie ostrzeżenia.

3.3 Kod programu

Listing 3.3: Program ostrzegający przed kolizją

```
1  import time
2  import math
3  from dronekit import connect, APIException
4
5  from gpiozero import Device, DistanceSensor
6  from gpiozero.pins.lgpio import LGPIOFactory
7
8  Device.pin_factory = LGPIOFactory()
9
10 DRONE_PORT = "/dev/serial/by-id/usb-Holybro_Pixhawk6C_450046001751333337363133-if0
11
12 MIN_DISTANCE_CM = 30
13 MIN_SPEED = 0.3 # w m/s
14 GAP = 0.05 # 50ms
15 MAX_DIST = 8
16
17 SENSOR_ANGLES_DEG = {
18     1: -45.0,
19     2: 0.0,
20     3: 45.0
21 }
22
23 sensor1 = DistanceSensor(echo=17, trigger=23, max_distance=MAX_DIST)
24 sensor2 = DistanceSensor(echo=27, trigger=24, max_distance=MAX_DIST)
25 sensor3 = DistanceSensor(echo=22, trigger=25, max_distance=MAX_DIST)
26
```

```
27
28 def connect_pixhawk():
29     print("Connecting to Pixhawk...")
30
31     while True:
32         try:
33             vehicle = connect(
34                 DRONE_PORT,
35                 baud=115200,
36                 wait_ready=True,
37                 heartbeat_timeout=60
38             )
39             print("Connected to Pixhawk")
40             return vehicle
41
42         except APIException as e:
43             print(f"No heartbeat yet: {e}")
44             time.sleep(3)
45
46         except Exception as e:
47             print(f"Connection failed: {e}")
48             time.sleep(3)
49
50
51 def get_body_velocity(vehicle):
52     velocity = vehicle.velocity
53     yaw = vehicle.attitude.yaw
54
55     if velocity is None or yaw is None:
56         return 0.0, 0.0
57
58     vn, ve, _ = velocity
59
60     v_forward = vn * math.cos(yaw) + ve * math.sin(yaw)
61     v_right = -vn * math.sin(yaw) + ve * math.cos(yaw)
62
63     return v_forward, v_right
64
65
66 def get_speed_toward_sensor(v_forward, v_right, sensor_angle_deg):
67     angle_rad = math.radians(sensor_angle_deg)
68     return v_forward * math.cos(angle_rad) + v_right * math.sin(angle_rad)
69
70
71 def send_alert(vehicle):
```

```
72     message = "obstacle"
73
74     vehicle.message_factory.statustext_send(
75         mavutil.mavlink.MAV_SEVERITY_ALERT,
76         message.encode("utf-8")
77     )
78     vehicle.flush()
79
80
81 def check_distance(vehicle, distance_cm, sensor_id, v_forward, v_right):
82     sensor_angle = SENSORANGLES_DEG[sensor_id]
83     toward_speed = get_speed_toward_sensor(v_forward, v_right, sensor_angle)
84
85     if distance_cm < MIN_DISTANCE_CM and toward_speed > MIN_SPEED:
86         send_alert(vehicle)
87
88
89 def main():
90     vehicle = None
91
92     try:
93         vehicle = connect_pixhawk()
94
95         vehicle.message_factory.statustext_send(
96             mavutil.mavlink.MAV_SEVERITY_ALERT,
97             b"Companion computer connected"
98         )
99         vehicle.flush()
100
101         while True:
102             v_forward, v_right = get_body_velocity(vehicle)
103
104             d1 = sensor1.distance * 100 * MAX_DIST
105             check_distance(vehicle, d1, 1, v_forward, v_right)
106             time.sleep(GAP)
107
108             d2 = sensor2.distance * 100 * MAX_DIST
109             check_distance(vehicle, d2, 2, v_forward, v_right)
110             time.sleep(GAP)
111
112             d3 = sensor3.distance * 100 * MAX_DIST
113             check_distance(vehicle, d3, 3, v_forward, v_right)
114             time.sleep(GAP)
115
116     except KeyboardInterrupt:
```

```

117         pass
118
119     finally:
120         if vehicle:
121             vehicle.close()
122
123
124 if __name__ == "__main__":
125     main()
126

```

3.4 Dynamiczne ustawianie odległości progu aktywacji

Program wyżej miał ustawioną na stałe odległość, poniżej której wykryta przeszkoda wywoła wysłanie powiadomienia. Na jego podstawie powstała bardziej rozwinięta wersja. Zamiast jednego progu aktywacji definiowanego z góry, Program odczytuje prędkość drona i ustala próg dynamicznie, dla każdego czujnika oddzielnie. W tym celu wykorzystano pomiary czasu reakcji i ustalono średni czas na 1.6 s oraz maksymalną odległość mierzoną przez czujniki na 4.5 m. Na podstawie tych danych obliczana jest odległość, z jakiej pilot będzie w stanie zareagować na zbliżającą się przeszkodę.

Listing 3.4: Program ostrzegający przed kolizjąaaaa

```

1  import time
2  import math
3  import csv
4  from datetime import datetime
5
6  from dronekit import connect, APIException
7  from pymavlink import mavutil
8
9  from gpiozero import Device, DistanceSensor
10 from gpiozero.pins.lgpio import LGPIIOFactory
11
12 Device.pin_factory = LGPIIOFactory()
13
14 DRONE_PORT = "/dev/serial/by-id/usb-Holybro_Pixhawk6C_450046001751333337363133-if0
15
16 REACTION_TIME_S = 1.6
17 GAP = 0.05 # 50 ms
18 MAX_DIST_M = 4.5 # w metrach
19 MAX_DIST_CM = MAX_DIST_M * 100.0

```

```
20 LOG_FILE = "sensor_threshold_log.csv"
21
22 SENSOR_ANGLES_DEG = {
23     1: -45.0,
24     2: 0.0,
25     3: 45.0
26 }
27
28 sensor1 = DistanceSensor(echo=17, trigger=23, max_distance=MAX_DIST_M)
29 sensor2 = DistanceSensor(echo=27, trigger=24, max_distance=MAX_DIST_M)
30 sensor3 = DistanceSensor(echo=22, trigger=25, max_distance=MAX_DIST_M)
31
32
33 def init_log_file():
34     try:
35         with open(LOG_FILE, "x", newline="") as f:
36             writer = csv.writer(f)
37             writer.writerow([
38                 "timestamp",
39                 "sensor_id",
40                 "distance_cm",
41                 "toward_speed_m_s",
42                 "threshold_cm"
43             ])
44     except FileExistsError:
45         pass
46
47
48 def log_sensor_data(sensor_id, distance_cm, toward_speed, threshold_cm):
49     with open(LOG_FILE, "a", newline="") as f:
50         writer = csv.writer(f)
51         writer.writerow([
52             datetime.now().isoformat(timespec="seconds"),
53             sensor_id,
54             f"{distance_cm:.2f}",
55             f"{toward_speed:.2f}",
56             f"{threshold_cm:.2f}"
57         ])
58
59
60 def connect_pixhawk():
61     print("łączenie z Pixhawk...")
62
63     while True:
64         try:
```

```
65         vehicle = connect(
66             DRONE_PORT,
67             baud=115200,
68             wait_ready=True,
69             heartbeat_timeout=60
70         )
71         print("połączono z Pixhawk")
72         return vehicle
73
74     except APIException as e:
75         print(f"brak heartbeat: {e}")
76         time.sleep(3)
77
78     except Exception as e:
79         print(f"nie połączono: {e}")
80         time.sleep(3)
81
82
83 def get_body_velocity(vehicle):
84     velocity = vehicle.velocity
85     yaw = vehicle.attitude.yaw
86
87     if velocity is None or yaw is None:
88         return 0.0, 0.0
89
90     vn, ve, _ = velocity
91
92     v_forward = vn * math.cos(yaw) + ve * math.sin(yaw)
93     v_right = -vn * math.sin(yaw) + ve * math.cos(yaw)
94
95     return v_forward, v_right
96
97
98 def get_speed_toward_sensor(v_forward, v_right, sensor_angle_deg):
99     angle_rad = math.radians(sensor_angle_deg)
100     return v_forward * math.cos(angle_rad) + v_right * math.sin(angle_rad)
101
102
103 def calculate_threshold_cm(toward_speed):
104     closing_speed = max(0.0, toward_speed)
105
106     threshold_cm = closing_speed * REACTION_TIME_S * 100.0
107
108     threshold_cm = min(threshold_cm, MAX_DIST_CM)
109
```

```
110     return threshold_cm
111
112
113 def send_alert(vehicle, sensor_id):
114     message = f"obstacle {sensor_id}"
115
116     vehicle.message_factory.statustext_send(
117         mavutil.mavlink.MAV_SEVERITY_ALERT,
118         message.encode("utf-8")[:50]
119     )
120     vehicle.flush()
121
122
123 def check_distance(vehicle, distance_cm, sensor_id, v_forward, v_right):
124     sensor_angle = SENSOR_ANGLES_DEG[sensor_id]
125     toward_speed = get_speed_toward_sensor(v_forward, v_right, sensor_angle)
126     threshold_cm = calculate_threshold_cm(toward_speed)
127
128     log_sensor_data(sensor_id, distance_cm, toward_speed, threshold_cm)
129
130     print(
131         f"Sensor {sensor_id}: "
132         f"dystans={distance_cm:.2f} cm, "
133         f"prędkość={toward_speed:.2f} m/s, "
134         f"próg={threshold_cm:.2f} cm"
135     )
136
137     if threshold_cm > 0.0 and distance_cm < threshold_cm:
138         send_alert(vehicle, sensor_id, distance_cm, threshold_cm)
139
140
141 def main():
142     vehicle = None
143
144     try:
145         init_log_file()
146         vehicle = connect_pixhawk()
147
148         vehicle.message_factory.statustext_send(
149             mavutil.mavlink.MAV_SEVERITY_ALERT,
150             b"Companion computer connected"
151         )
152         vehicle.flush()
153
154         while True:
```

```
155         v_forward, v_right = get_body_velocity(vehicle)
156
157         d1 = sensor1.distance * 100.0
158         check_distance(vehicle, d1, 1, v_forward, v_right)
159         time.sleep(GAP)
160
161         d2 = sensor2.distance * 100.0
162         check_distance(vehicle, d2, 2, v_forward, v_right)
163         time.sleep(GAP)
164
165         d3 = sensor3.distance * 100.0
166         check_distance(vehicle, d3, 3, v_forward, v_right)
167         time.sleep(GAP)
168
169     except KeyboardInterrupt:
170         pass
171
172     finally:
173         if vehicle:
174             vehicle.close()
175
176
177 if __name__ == "__main__":
178     main()
179
```

3.5 Testy

W celu weryfikacji działania zbudowanego systemu wykonano szereg testów w trakcie rozwoju, które potem powtórzono po ukończeniu prac. Testy obejmowały zarówno sprawdzenie działania i parametrów użytego sprzętu, jak również poprawność działania napisanych programów.

3.5.1 Sprawdzenie poprawności działania systemu

Czujniki odległości zostały przetestowane poprzez ustawienie płaskiej przeszkody prostopadle do czujnika, w odległościach 30 cm, 50 cm, 1 m, 2 m i 4 m. We wszystkich przypadkach wykonane pomiary nie odbiegały od rzeczywistej odległości powyżej 1 cm różnicy, co jest wystarczającą dokładnością dla wykrywania przeszkód. W celu sprawdzenia działania programu biorącego pod uwagę prędkość zbliżania się do przeszkody wykonano imitację przeszkody montowaną na BSP kilka centymetrów przed

jednym z czujników odległości. Dron następnie był wznoszony w powietrze i rozpędzany do prędkości przekraczającej minimalną wymaganą do wysłania ostrzeżenia. Próbę powtórzono dla wszystkich trzech czujników.

Połączenie między komputerem pokładowym i asystującym jest weryfikowane poprzez wysłanie komunikatu o pomyślnym otwarciu połączenia. Komunikat ten może być odczytany przy użyciu komputera z aplikacją Mission Planner.

3.5.2 Pobór prądu

Pobór prądu Raspberry Pi w trakcie działania programu został zmierzony z użyciem miernika KWS-2302C. Pomiary wahały się pomiędzy 3.371 W a 3.612 W, w większości oscylując około 3.491 W. Biorąc pod uwagę parametry akumulatora zasilającego komputer asystujący można oszacować czas pracy na jednym ładowaniu na około 10.5 godziny.

3.5.3 Dokładność pomiarów odległości

Czujniki odległości HC-SR04, pomimo tego samego oznaczenia, w zależności od producenta mogą różnić się parametrami takimi jak maksymalna odległość wykonywania pomiarów. Użycie ultradźwięków wiąże się też z problemami z wykrywaniem przeszkód ustawionych pod kątem, fala dźwiękowa odbita od przeszkody może nigdy nie wrócić do czujnika. W związku z tym wykonano testy z użyciem czujników wykorzystanych w projekcie. Czujniki były w stanie zarejestrować przeszkodę ustawioną prostopadle z maksymalnie 460 cm, powyżej tej odległości odczyty były niestabilne i większość pomiarów kończyła się niewykryciem wracającej fali ultradźwiękowej. Sprawdzone również wpływ nachylenia przeszkody od pozycji prostopadłej do czujnika na poprawność odczytów. Pomiary wykonano w odległościach: 50 cm, 1 m, 2 m i 3 m. Pomiary wykazały:

- 50 cm — odczyty poprawne niezależnie od kąta odchylenia przeszkody,
- 1 m — odczyty niepoprawne przy odchyleniu $>50^\circ$,
- 2 m — odczyty niepoprawne przy odchyleniu $>22^\circ$,
- 3 m — odczyty niepoprawne przy odchyleniu $>12^\circ$.

3.5.4 Czas reakcji systemu

Przy wykrywaniu przeszkód przed poruszającym się pojazdem kluczowe jest odpowiednio niskie opóźnienie, tak aby osoba kierująca miała wystarczający czas na reakcję.

W celu pomiaru opóźnień wykorzystano odpowiednio zmodyfikowany program 3.3. Na potrzeby pomiaru zmieniono warunki aktywacji systemu tak, aby pomiary pierwszego i drugiego czujnika nie spełniały warunków potrzebnych do wysłania powiadomienia. Warunek dla trzeciego czujnika został zmodyfikowany tak by był on spełniany za każdym razem. Dzięki takim ustawieniom pomiary wykonywane były przy założeniu najbardziej pesymistycznego scenariusza, gdzie przeszkoda pojawia się w idealnym momencie by wymagała przejścia przez pozostałe czujniki zanim zostanie wykryta. Program rozpoczynał pomiar czasu po ustanowieniu połączenia z komputerem pokładowym i przed rozpoczęciem pomiarów odległości. W pierwszej wersji programu pomiar rozpoczynany był ręcznie poprzez naciśnięcie klawisza. Klawisz był następnie naciskany ponownie w momencie usłyszenia ostrzeżenia odczytanego przez komputer z aplikacją Mission Planner połączoną z dronem, co kończyło pomiar i zapisywało czas. W drugiej wersji programu ręczne uruchamianie pomiaru zostało zastąpione uruchamianiem automatycznym po losowym czasie 5–20 sekund. Wprowadzenie nieprzewidywalności do pomiaru pozwoliło na pomiar opóźnienia włącznie z ludzkim czasem reakcji, co w tym przypadku jest bardziej przydatne od opóźnień samego systemu. Wyniki obu pomiarów zostały przedstawione w tabeli 3.1

Tabela 3.1: Pomiary czasu pomiędzy wykryciem przeszkody a reakcją użytkownika

Rozpoczynane ręcznie [s]	Rozpoczynane losowo [s]
1.078	1.655
0.905	1.399
6.770	1.493
1.339	1.388
1.014	1.525
0.862	1.289
0.862	1.400
0.775	1.482
0.884	1.380
0.710	1.519

3.6 Alternatywne rozwiązania

W trakcie pracy nad systemem zapobiegającym kolizjom powstały alternatywne rozwiązania i pomysły które nie trafiły do końcowej wersji. Przykładowo program biorący pod uwagę prędkość drona przy decydowaniu o wysłaniu ostrzeżenia został napisany na podstawie wersji sprawdzającej jedynie odległość od przeszkody. W zależności

od preferencji użytkownika uzależnienie ostrzeżeń od prędkości może być niepożądane.

Problem z różnicą napięć między stanem wysokim dla czujników odległości a stanem wysokim dla pinów GPIO w Raspberry Pi może być rozwiązany na inne sposoby niż z życiem konwertera stanów logicznych. Istnieje możliwość stworzenia dzielnika napięcia z pomocą dwóch odpowiednio dobranych rezystorów. Z powodu braku takich pod ręką ta możliwość nie została jednak przetestowana. Podczas prac, aby przyspieszyć budowę platformy i przejść do oprogramowania, czujniki odległości zostały podłączone do Arduino Nano. Wywoływanie pomiarów i obliczanie na ich podstawie zmierzonej odległości odbywało się na mikrokontrolerze, a wyniki przesyłane były do Raspberry Pi dzięki połączeniu USB. Takie rozwiązanie wprowadza jednak niepotrzebnie dodatkowe urządzenie do układu oraz zwiększa jego zużycie energii przez konieczność zasilania mikrokontrolera. W przypadku gdy Raspberry Pi będzie działać z niedostatecznym zasilaniem może ograniczyć prąd idący do urządzeń połączonych przez USB do 600mA [6], w przypadku Arduino Nano takie ograniczenie nie stanowi jednak problemu ponieważ prąd wymagany do jego działania jest znacznie niższy. Użycie Raspberry Pi oraz Arduino komplikuje pracę nad oprogramowaniem; Do działającego systemu trzeba napisania dwóch programów w dwóch różnych językach, oddzielnie dla obu z urządzeń. Z powodu wymienionych wad użycia Arduino zostało ono zastąpione konwerterem stanów logicznych w późniejszych etapach rozwoju, pozostaje on jednak możliwym rozwiązaniem.

Użyte w tym przypadku ultradźwiękowe czujniki odległości nie są idealnymi urządzeniami pomiarowymi. Wykorzystanie fali dźwiękowej odbijającej się od przeszkody do określenia odległości wiąże się z ograniczeniami i podatnościami. Fala dźwiękowa, padająca na przeszkodę, odbija się różnie w zależności od jej materiału i kształtu. Dobrze wygłuszający materiał taki jak grube płótno czy gąbka może zostać niewykryty przez czujnik ultradźwiękowy. Czujniki wykorzystujące ultradźwięki mają również problem z wykrywaniem przeszkód ustawionych pod zbyt dużym kątem. Idealne warunki pomiarowe to przeszkoda ustawiona prostopadle do czujnika. Naukowcy Manabu Ishihara, Makoto Shiina i Shin-nosuke Suzuki przeprowadzili szereg eksperymentów z użyciem czujnika ultradźwiękowego [7] i odkryli że odległość od przeszkody ma wpływ na dopuszczalny kąt pomiaru. Wraz ze skracaniem dystansu można pozwolić sobie na coraz większy kąt i zachowanie niezawodności czujnika. Alternatywą dla czujników ultradźwiękowych mogą być moduły oparte o lasery podczerwone. Ponieważ wiązka lasera porusza się z prędkością światła, znacznie szybciej niż prędkość fali dźwiękowej, pomiary laserem są o wiele szybsze od tych wykonywanych przez czujnik ultradźwiękowy. Używanie wielu czujników laserowych nie wiąże się też z ryzykiem zaburzania odczytów przez siebie nawzajem, co w praktyce eliminuje potrzebę ustawiania opóźnień między pomiarami. Z drugiej strony użycie lasera sprawia że obiekty szklane

lub odbijające światło nie zostaną poprawnie wykryte. Czujniki wykorzystujące laser podczerwony są też zazwyczaj droższe niż czujniki ultradźwiękowe. Zastosowanie czujników HC-SR04 motywowane było głównie ich niską ceną oraz dostępnością, jednak wymienione wcześniej zalety czujników laserowych czynią z nich wartą przemyślenia alternatywę.

Rozdział 4

Potencjalne ścieżki rozwoju projektu

- 4.1 Półautonomiczny system zapobiegania kolizjom
- 4.2 Wielokierunkowy monitoring
- 4.3 Sieć neuronowa rozpoznawanie obiektów z obrazu kamery
- 4.4 mapa 3d lidar

Bibliografia

- [1] ArduPilot. Mission planner overview, 2026. dostę: 13.02.2026.
- [2] EASA. Drone regulatory system, 2026. dostę: 11.02.2026.
- [3] Holybro. Air2216ii, blheli s 20a, t1045ii documentation. Technical report, 2026. dostę: 13.02.2026.
- [4] Holybro. M10 gps module, 2026. dostę: 13.02.2026.
- [5] Raspberry Pi Ltd. Raspberry pi 5 release article, 2026. dostę: 27.04.2026.
- [6] Raspberry Pi Ltd. Raspberry pi documentation, 2026. dostę: 26.04.2026.
- [7] Shin-nosuke Suzuki Manabu Ishihara, Makoto Shiina. Evaluation of method of measuring distance between object and walls using ultrasonic sensors. *Journal of Asian Electric Vehicles*, 7(1):1207–1211, June 2009.
- [8] PX4. Pixhawk 6c overview, 2026. dostę: 13.02.2026.
- [9] Cytron Technologies. Hc-sr04 user manual, 2013. dostę: 11.05.2026.